

## Relatório Programação Orientada a Objetos

|   | A | B | C | D | E | F | G | H | I | J |
|---|---|---|---|---|---|---|---|---|---|---|
| A | c |   |   | c |   |   |   |   |   |   |
| B |   |   |   |   |   |   |   |   |   |   |
| C |   |   |   | e |   |   |   |   |   |   |
| D |   |   |   |   |   |   |   |   |   |   |
| E |   |   |   |   | * |   |   |   |   |   |
| F |   |   |   |   |   |   |   |   |   |   |
| G |   |   |   |   |   |   |   |   |   |   |
| H |   |   |   |   |   |   |   |   |   |   |
| I |   |   |   |   |   | x |   |   | r |   |
| J |   |   |   |   |   |   |   |   |   |   |

Trabalho realizado por

- Diogo Filipe da Silva Pinto 2024152575
- Rafael Tavares Marques 2024143044

## Índice

|                                         |   |
|-----------------------------------------|---|
| Índice                                  | 2 |
| Introdução                              | 3 |
| Opções Tomadas e Estrutura de Dados     | 3 |
| Estrutura da Dados da Grelha do Jardim  | 3 |
| Gestão de Entidades no Solo (Ownership) | 4 |
| Representação e Prioridade Visual       | 4 |
| Arquitetura de Classes                  | 5 |
| Descrição das classes                   | 6 |
| Classe Gestora de Comandos              | 6 |
| Classe Gestora do Ambiente              | 6 |
| Hierarquia de Classes                   | 6 |
| Validação de Comandos                   | 7 |
| Fluxo de Processamento dos Comandos     | 7 |
| Conclusão                               | 8 |

## Introdução

O presente relatório tem como objetivo apresentar o desenvolvimento do trabalho prático da unidade curricular POO (Programação Orientada a Objetos). O objetivo central do projeto é a conceção e implementação de um simulador de um Jardim. Este simulador é representado por uma grelha retangular onde um utilizador, na figura de 'jardineiro' pode interagir com o ambiente, gerir as diferentes espécies de plantas e utilizar as ferramentas. Toda a simulação evolui através de 'instantes' dados por ordem do utilizador.

## Opções Tomadas e Estrutura de Dados

A implementação do simulador exigiu duas decisões estruturais fundamentais, como armazenar a grelha do jardim e a gestão as entidades (plantas, ferramentas, jardineiro) dentro da grelha.

### Estrutura da Dados da Grelha do Jardim

Para o armazenamento das posições de solo do jardim utilizamos uma matriz alocada dinamicamente, representada por um apontador de apontadores (*Solo\*\* solo*).

No construtor da classe **Jardim**, primeiramente é alocado um array de linhas apontadores (`new Solo*[linhas]`) para depois subsequentemente cada índice desse array alocado, conter um array de colunas (`new Solo[colunas]`).

Tomamos essa decisão por ser uma tradução 'direta' de uma matriz 2D que permite o acesso intuitivo e legível às posições do solo, dado á inibição no uso da biblioteca standard (*std::vector*).

## Gestão de Entidades no Solo (Ownership)

A classe Solo centraliza as informações de cada célula do jardim, e a gestão das entidades que podem ocupar uma célula é feita através de apontadores, permitindo assim que cada posição se encontre vazia(`nullptr`) ou contenha uma entidade.

A posição do solo é considerada 'dona' da planta que contém, e como tal o destrutor da classe Solo é responsável por libertar memória da planta. A posição do Solo não é 'dona' do jardineiro, apenas armazena uma referência para o objeto Jardineiro gerido pela classe Jardim, o que evita a duplicação libertação de memória.

A gestão de ferramentas é de certo modo similar á das plantas, a posição de solo é 'dona' da ferramenta que contém, embora ainda não esteja aplicado o destrutor do solo será também responsável pela destruição da ferramenta.

## Representação e Prioridade Visual

A representação visual do jardim obedece á impressão 'para a frente e para baixo' facilitando a representação visual das entidades no jardim obedece à ordem de prioridade definida no enunciado (Jardineiro > Planta > Ferramentas). A implementação desta ordem visual foi implementada na classe Solo através do método *getSimbolo()*, que verifica a ordem correta de qual entidade deva ser representada no ecrã.

## Arquitetura de Classes

- Planta
  - Cacto
  - ErvaDaninha
  - PlantaExotica
  - Roseira
- Ferramenta
  - EnxadaRotativa
  - PacoteAdubo
  - Regador
  - TesouraPoda
- Comandos
  - Apaga
  - Avanca
  - Colhe
  - Compra
  - Entra
  - Executa
  - Fim
  - Larea
  - Larga
  - Lferr
  - Lplanta
  - LSolo
  - Pega
  - Planta
  - Sai

## Descrição das classes

A arquitetura do simulador foi planeada em torno de três tipos principais de classes, que separam as responsabilidades de interpretação de comandos, gestão do ambiente de simulação e a representação das entidades.

### Classe Gestora de Comandos

Esta classe é responsável por receber a mensagem do utilizador e traduzi-la em ações. A validação de comandos foi implementada de forma descentralizada, tirando partido da arquitetura '*Command Pattern*' da classe *GestorCmd*.

A classe *GestorCmd* recebe uma string, divide-a e encontra o objeto comando a que corresponde para executar.

### Classe Gestora do Ambiente

A classe *Jardim* é a classe 'contentora' principal responsável por alocar, gerir e destruir a grelha 2D do Solo.

### Hierarquia de Classes

Estas classes usam herança e polimorfismo para representar os objetos dinâmicos do jardim.

- Planta (Classe Base Abstrata)

Define o 'contrato' polimórfico para todas as espécies de plantas, tem como classes derivadas o *Cacto*, a *Roseira*, a *ErvaDaninha* e a *PlantaExotica*.

- Ferramenta (Classe Base Abstrata)
- Define o 'contrato' polimórfico para todas as ferramentas, tem como classes derivadas o *Regador*, o *PacoteAdubo*, a *TesouraPoda*, e a *EnxadaRotativa*.

## Validação de Comandos

A responsabilidade da validação não foi centralizada no *GestorCmd*, mas sim delegada a cada classe do comando individual (*CmdAvanca*, *CmdPlanta*, ...).

## Fluxo de Processamento dos Comandos

- O *main* captura a linha de texto completa do utilizador
- A linha de texto é passada para a função *intreperatar()* do *GestorCmd*.
- O *GestorCmd* usando uma função implementada nas **Utilidades**, divide a linha em segmentos, e encontra o objeto **Comando** cujo *getnome()* corresponde a primeira linha
- Após separado e encontrado o objeto (Comando), o *GestorCmd* chama o método *executar(segmentos, n)* do objeto, passando-lhe esses argumentos.

## Conclusão

A realização da Meta 1 do projeto permitiu-nos estabelecer os alicerces fundamentais no Simulador do Jardim, cumprindo com os dois objetivos centrais desta meta 1, planeamento da arquitetura e a validação dos comandos.

O requisito de leitura e validação dos comandos foi implementado com sucesso através de uma arquitetura baseada no 'Padrão de Comando'. O uso da classe **GestorCmd** como intérprete permitiu-nos fazer a lógica das validações de sintaxe e de parâmetros nas classes de comando individuais (*CmdPlanta*, *CmdCompra*, ...) resultando num sistema coeso e facilmente extensível.

Foi também concluído o planeamento e estruturação da arquitetura orientada a objetos. As classes foram devidamente separadas por responsabilidade.

O projeto encontra-se devidamente organizado em ficheiros *.h* e *.cpp*, com uma base de código estável e com as principais estruturas de dados definidas.

Concluídos todos os requisitos da meta 1, implementamos os comandos (não só as verificações como o seu devido funcionamento) de criação de jardim, entrada do jardineiro no jardim, movimentações do jardineiro no jardim, saída do jardineiro, e o comando planta, o comando colhe, todos os comandos de listagem (*Lplanta*, *Lcolhe*, *Lplantas*, *Larea*, *Lsolo*, *Lferr*) e o comando fim (liberta a memória utilizada).